CPDS Laboratory Assignment 2: Parallel Programming with OpenMP Solving the Heat Equation

José R. Herrero

Fall 2026



Acknowledgment of past contributions: E. Ayguadé and D. Jiménez

Index

Index **1**

1 Before starting this assignment ... **2**
1.1 Task versus data decomposition strategies . . . 2
1.2 Some data decomposition strategies . . . 2

2 Sequential Jacobi and Gauss-Seidel heat diffusion codes **4**

3 Dependency analysis **6**

4 Parallel Jacobi **7**
4.1 Data Decomposition Implementation . . . 7
4.1.1 Overall Analysis . . . 7
4.1.2 Detailed Analysis . . . 8
4.1.3 Optimization . . . 8

5 Parallel Gauss–Seidel **9**
5.1 Task decomposition: Implementation with `explicit tasks with dependences` . . . 9
5.2 Data decomposition: Implementation with `do-across` . . . 10
5.3 Detailed Analysis and Optimization . . . 10
5.3.1 Finding the appropriate value for the number of blocks . . . 10
5.3.2 Overall Analysis . . . 11
5.4 Memory analysis . . . 11

6 Deliverable **12**

7 Annex 1: Creating your own synchronisation objects **13**

Note:

- All files necessary to do this laboratory assignment are available in a compressed tar file available from the following location: `/scratch/nas/1/par0/sessions/a2.tar.gz`. Uncompress it into your home directory in `boada.ac.upc.edu` with this command line:
`"tar -zxvf /scratch/nas/1/cpds0/sessions/a2.tar.gz"`.
- The Makefile and the shell scripts assume certain file names. It's more efficient for you to always use those filenames for the current parallel version you're working on, keeping copies of other versions separately using different meaningful names. Also, in the deliverable you'll be asked to submit those files containing the parallel C source code.

1

Before starting this assignment ...

The goal of this assignment is comparing the usage of task and data decompositions with OpenMP for solving the heat diffusion equation. Before starting this laboratory assignment, we strongly recommend that you take a look at this section and try to solve the simple questions we propose to you. This will help to better face this programming assignment in OpenMP.

1.1 Task versus data decomposition strategies

In this laboratory assignment you will explore both decomposition strategies: *task decomposition* using **explicit tasks** and *data decomposition* making use of **implicit tasks**. The tasking model in OpenMP is enabled by OpenMP clauses `task` and `taskloop`. Making use of explicit tasks is very versatile, allowing us to express a wide range of dynamic task decompositions, both iterative and recursive. However, **using explicit tasks the programmer has no control on *data locality*.**

1.2 Some data decomposition strategies

With a data decomposition strategy the computation that each implicit task has to perform is determined by the data it has to access, either read or write. This may have clear benefits in terms of data locality exploitation, because each thread will always execute implicit tasks that access to the same data, whenever possible. *Data decompositions* can be *Geometric* or *Recursive*. In this assignment we will focus our attention on the Geometric ones that are applied to n-dimensional matrices (including vectors).

Assume the code shown in Figure 1.1 for which we want to write a parallel code ensuring that each thread will compute the elements of matrix `C` that are stored in its own memory.

```
void vectoradd(int *A, int *B, int *C, int N) {
   int i, j;

   for (i=0; i< N; i++)
       for (j=0; j< N; j++)
           C[i*N+j] = A[i*N+j] + B[i*N+j];

}
```

Figure 1.1: Simple example performing matrix sum.

Assuming that P is the number of processors and that $thread_k$ is executed on processor P_k, the left part of Figure 1.2 shows a possible **block distribution** for the matrices by rows, so that the memory associated to each processor has a block of consecutive $N \div P$ rows of matrices `A`, `B` and `C`. On the right we show a parallel implementation using implicit tasks for the loop that follows the so called *Owner-computes Rule*: each processor is responsible for the computations on the elements that are allocated in its main memory. Observe that, based on the value returned by intrinsic `omp_get_thread_num()`, each thread can determine the subset of iterations of the `i` loop to execute (range between `i_start` and

i_end); since columns are not distributed, each thread executes all iterations of the j loop. For simplicity, the code in Figure 1.2 assumes that the number of elements N is a multiple of the number of threads executing the parallel region; think how it would change if this condition does not hold (think about it, trying to maximise load balancing).

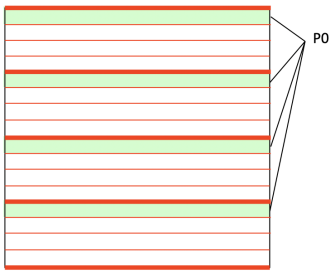



```
#pragma omp parallel private(i, j)
{
    int my_id = omp_get_thread_num();
    int BS = N/omp_get_num_threads();
    int i_start = my_id * BS;
    int i_end = i_start + BS;

    for (i=i_start; i<i_end; i++)
        for (j=0; j<N; j++)
            C[i*N+j] = A[i*N+j] + B[i*N+j];
}
```

Figure 1.2: Left: geometric block data decomposition by rows. Right: parallelisation using implicit tasks.

Similarly, Figures 1.3 and 1.4 show two different data decomposition strategies and the code associated, always using implicit tasks, that follows the owner computes rule. In Figure 1.3 each processor also has $N \div P$ rows but now distributed in a **cyclic** way, starting with row 0 assigned to thread 0. Observe that the loop i now traverses the iteration space starting from the identifier of the thread executing the implicit task, jumping as many iterations as threads in the parallel region until reaching N.

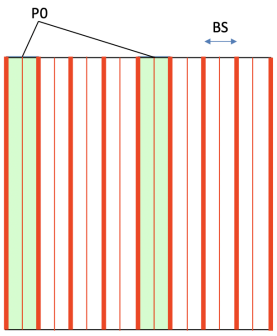



```
#pragma omp parallel private(i, j)
{
    int my_id = omp_get_thread_num();
    int howmany = omp_get_num_threads();

    for(i=my_id; i<N; i+=howmany)
        for (j=0; j<N; j++)
            C[i*N+j] = A[i*N+j] + B[i*N+j];
}
```

Figure 1.3: Left: geometric cyclic data decomposition by rows. Right: parallelisation using implicit tasks.

Figure 1.4 shows a **block-cyclic distribution by columns**: each processor has blocks of $BS = 2$ consecutive columns assigned in a cyclic way, starting with the first block assigned to thread 0. The loop that is decomposed is the j loop, creating a column distribution since j is the index used to access the columns. Note that there are 3 loops instead of 2 as a result of applying *Strip-Mining*[1].




```
#pragma omp parallel private(i, j)
{
    int my_id = omp_get_thread_num();
    int howmany = omp_get_num_threads();
    for(i=0; i<N; i++)
        for (jj=my_id*BS; jj<N; jj+=howmany*BS)
            for (j=jj; j<jj+BS; j++)
                C[i*N+j] = A[i*N+j] + B[i*N+j];
}
```

Figure 1.4: Left: geometric block-cyclic data decomposition by columns. Right: parallelisation using implicit tasks, assuming $N\%(BS \times howmany) == 0$.

[1]Strip-Mining: a loop is transformed into a depth-2 loop nest, where the outer loop is called tile/block loop and the innermost loop is called element loop.

2

Sequential Jacobi and Gauss-Seidel heat diffusion codes

In this laboratory assignment you will work on the parallelisation of a sequential code that simulates the diffusion of heat in a solid body using two different solvers for the heat equation (*Jacobi* and *Gauss-Seidel*). Each solver has different numerical properties which are not relevant for the purposes of this laboratory assignment; we use them because they show different parallel behaviours. In any case, you should be familiar with the two solvers since we have been using them in the course.

Take a look at the the source code of `heat-seq.c` (where the solver is invoked) and `solver-seq.c` (where the solvers are coded). You will soon realise that both solvers use the same function `solve`. The difference is that *Jacobi* uses a temporary matrix to store the new computed matrix (`param.uhelp`) while *Gauss-Seidel* directly updates the same matrix (`param.u`). Notice that function `solve` is iteratively invoked inside a `while` loop that iterates while two different conditions are met: 1) the maximum number of iterations `param.maxiter` is not reached; and 2) the value returned by the solver is larger than `param.residual`. And also that at each iteration of the `while` loop *Jacobi* needs to copy the newly computed matrix into the original one in order to repeat the process, by simply invoking funtion `copy_mat` also defined inside `solver-seq.c`.

Note: Recall that the dependencies arising in some parallel codes code require the application of *blocking* in order to achieve *wavefront parallelism*. To this end, the sequential code is already prepared to work by blocks. For this reason, and contrary to the course slides, the codes implementing the numerical solver provided already have 4 loops. Handling more loops has additional overhead, and is not the most efficient in terms of execution time when not needed. Consequently, you can use a simpler version with 2 loops in those cases which do not need *blocking*. Thus, the skeleton of the OpenMP Jacobi implementation has 2 loops.

The picture in Figure 2.1 shows the resulting heat diffusion when two heat sources are placed in the borders of the 2D solid (one in the upper left corner and the other in the middle of the lower border). The program is executed with a configuration file (`test.dat`) that specifies the number of heat sources, their position, size and temperature. The program also accepts several execution arguments: `-n`: maximum number of simulation steps or iterations; `-s`: the size of the body (resolution); `-r`: the residual value that stops the algorithm; `-a`: the solver to be used; and `-o` the output file (an image similar to the one shown in Figure 2.1 in portable pixmap file format, showing a gradient from red (hot) to dark blue (cold)). The execution of the program reports the execution time and performance measurements.

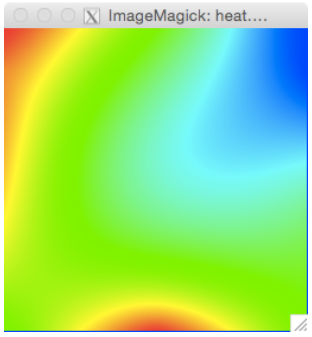


Figure 2.1: Image representing the temperature in each point of the 2D solid body

1. Compile the sequential version of the program using `"make heat-seq"` and execute the binary generated using the *Jacobi* solver: `"./heat-seq test.dat -a 0 -o heat-jacobi-seq.ppm"`. The execution reports the execution time (in seconds), the number of floating point operations (Flop) performed, the average number of floating point operations performed per second (Flop/s), the residual and the number of simulation steps performed to reach that residual. Visualise the image file generated with an image viewer (e.g. `"display heat-jacobi-seq.ppm"`); keep this file in your directory to check later the correctness of the parallel versions you will program.

2. Change the solver from *Jacobi* to *Gauss-Seidel* by simply re-executing with `"./heat-seq test.dat -a 1 -o heat-gauss-seq.ppm"`. Observe the differences with the previous execution. Note: the images generated when using the two solvers are slightly different (you can check this by applying `diff` to the two image files generated). Again, keep the `.ppm` file generated to check later the correctness of the parallel versions you will program.

3. Keep the resulting `.ppm` files generated with the original sequential code, which is regarded as a correct baseline. Subsequent parallel executions in the next sections will also generate image files. You'll need the original images for validating the correctness of the parallel implementations, by comparing the resulting images. Once your reach that point, and since the images generated by the two numerical methods are slightly different, remember to compare against the image generated by the same solver, either jacobi or gauss-seidel, with the original sequential code.

3

Dependency analysis

Using a tool named `Tareador` we want to analyze the potential parallelism that can be achieved by different *task decomposition* strategies. `Tareador` traces the execution of the program based on the specification of potential tasks to be run in parallel. It records all static/dynamic data allocations and memory accesses in order to build the *task dependency graph (TDG)*. Additionally, it offers the possibility to simulate the parallel execution of the tasks on a chosen number of processors in order to estimate the potential *speedup*.

Tareador's application programmer interface (API) allows to specify code regions *to be considered* as potential tasks, where the string `TaskName` identifies that task in the graph produced by Tareador:

```
tareador_start_task("TaskName");
/* Code region to be a potential task */
tareador_end_task("TaskName");
```

In order to enable the analysis with Tareador, the programmer must invoke:

```
tareador_ON();
/* Main program */
tareador_OFF();
```

at the beginning and at the end of the program, respectively.

We provide two source files named `heat-tareador.c` and `solver-tareador.c` which include the definitions of some tasks. They also have some other definitions commented out. The `Makefile` has rules to use them to generate an executable called `heat-tareador`. However, this file cannot be executed directly. For this reason we provide a shell script which can be used for both compiling the code and executing it on a small problem in a special environment needed by `Tareador`. To this end, you only need to execute the command `./run-tareador.sh program options`. Specifically,

- `./run-tareador.sh heat-tareador 0` works on the *Jacobi* version, while
- `./run-tareador.sh heat-tareador 1` works on the *Gauss-Seidel* version.

We can disable the analysis of certain variables in the program using the following functions of the API:

```
tareador_disable_object(&name_var)
/* Code region with memory accesses to variable name_var */
tareador_enable_object(&name_var)
```

With this mechanism you remove all the dependencies caused by the selected variable in a specific code region. We have used this mechanism to disable the tracking of variable `sum` which cause a *loop carried dependency* so that we can observe if other dependencies exist.

Commenting and uncommenting tasks we invite you to observe the potential of different task decompositions for both Jacobi and Gauss-Seidel.

For the deliverable: Include the *TDG* and simulation with *enough* processors for those versions which you have explored for both Jacobi and Gauss-Seidel. Please, make sure to clearly identify which version of the code was used to obtain each TDG and corresponding simulated parallel execution.

4

Parallel Jacobi

In this chapter you will parallelise the sequential code for the heat equation code considering the use of the *Jacobi* solver, using the **implicit tasks** generated in a `#pragma omp parallel`[1], following a *geometric block data decomposition by rows*, as shown in Figure 4.1 for 4 threads running on 4 processors.

4.1 Data Decomposition Implementation

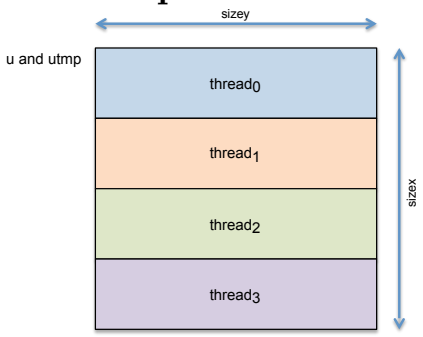


Figure 4.1: Geometric (data) decomposition for matrix `u` (and `utmp`) by rows, for 4 threads.

With this strategy the computation that the implicit task executed each thread has to perform is determined by the data it has to access, either read or write. This may have clear benefits in terms of data locality exploitation because each thread will always execute implicit tasks that access to the same data, whenever possible. How could we parallelise the sequential code for function `solve` to ensure that each processor computes its assigned elements?

For a start, analyse the initial parallelisation proposed in `solver-omp.c` for function `solve`. Try to understand how the proposed parallelisation follows the data decomposition strategy mentioned above. Is the code correct? **Complete the parallel code so that it honors the dependences that you have discovered when analyzing the *Jacobi* solver**. Compile using `make heat-omp` and submit its execution to the queue using the `submit-omp.sh` script, specifying the binary file, the use of the Jacobi solver and 1 (`sbatch submit-omp.sh heat-omp 0 1`) and 8 threads (`sbatch submit-omp.sh heat-omp 0 8`). Validate the parallelisation by visually inspecting the image generated and making a `diff` with the file generated with the original sequential version. **Please**, if the image is not similar, do not continue to the next section since it'd be a waste of your time and the computer resources. Compare also the number of iterations until convergence.

4.1.1 Overall Analysis

Once the parallelisation is validated, submit the execution of the `submit-strong-extrae.sh` script to trace the execution for different number of threads (1, 4, 8 and 16). This script will also execute `mfLite.py` that will help you to perform the overall analysis. When submitting the script you should specify the solver to be used (argument with value 0). In order to do a better analysis of scalability, this script is already modified to run with a larger problem size (-s 1022) but less

[1] **Important:** You can not make use of other pragmas to create explicit tasks or distribute the work among the implicit tasks. The goal is that you use OpenMP intrinsic functions and explicitly control the work assigned to each thread.

simulation steps (-n 1000). The submission must be done as: `sbatch ./submit-strong-extrae.sh heat-omp 0` After the execution finishes several files have been created. Inspect the contents of subdirectory `heat-omp-jacobi-strong-extrae`. A summary of the results can be found in a text file named `modelfactors.out`, or a pdf file named `modelfactor-tables.pdf` which can be inspected with the `xpdf` command.
For the deliverable: Include the *modelfactors* tables. Is the scalability that is obtained with this initial parallelisation appropriate? Which is the metric reported by `mfLite.py` that you should address first?

4.1.2 Detailed Analysis

We recommend you to open with `paraver` the trace that has been generated for 16 threads and observe what is causing the low value for that metric. Use `paraver` Hints Instantaneous Parallelism and Implicit Tasks in Parallel Constructions, synchronize them and zoom in one of the trace to see the detail of the execution. To this end, execute the following command:
`wxparaver heat-omp-jacobi-strong-extrae/heat-omp-16-boada-18-cutter.prv`, replacing boada-18 with the one assigned to your execution.
For the deliverable: Include some captures of the window timelines to show the problem. What is the region of the code that is provoking the low value for the *parallel fraction* in your parallelisation? Hint: What needs to be done in between subsequent iterations of the Jacobi solver? Remember the *Tareador* analysis you did.

4.1.3 Optimization

Parallelise other parts of the code in order to improve the efficiency of your parallel code. Compile the new version and submit its execution to the queue using `submit-omp.sh`, specifying the binary file, the use of the Jacobi solver and 16 threads. Make sure the new parallel version still generates correct results.
For the deliverable: Include a code excerpt to show the OpenMP annotations you have added.

Overall Analysis of the Optimized Code

Once correctness is validated, submit again the `submit-strong-extrae.sh` script and analyse the scalability of the new parallelisation for different number of threads (1, 4, 8 and 16); do not forget to specify with an argument the solver to be used (0). For completeness, use the `submit-strong-omp.sh` script to queue the execution of `heat-omp` and obtain the scalability plot to be included in the deliverable; do not forget to specify with an argument the solver to be used (0). Unlike in assignment 0, we use the `time` command to measure elapsed execution time. It's precision is very poor. Consequently performance can show superlinear speedups due to the lack of precision.
For the deliverable: Include the *modelfactors* tables and the plot of scalability. Is the execution time reduced?. Have you increased the scalability? Is the *parallel fraction* larger than before? What is the speedup that you have achieved compared to your first implementation for 16 threads?

Detailed Analysis of the Optimized Code

We recommend you to open with `paraver` the trace that has been generated for 16 threads and observe the behaviour for your parallel code. Use `paraver` Hints Instantaneous Parallelism and Implicit Tasks in Parallel Constructions, synchronize them and zoom in one of the trace to see the detail of the execution.
For the deliverable: Include some captures of the window timelines.

Improved Optimized Code

Above, we suggested parallelizing some additional part of the execution which was originally performed sequentially in between iterations of the Jacobi solver. However, instead of parallelizing the code, some other optimization could be done. Hint: The solver routine just needs the initial addresses of the input and output matrices.
For the deliverable: Include some statistics and captures of the window timelines if this optimization is implemented.

5

Parallel Gauss–Seidel

Once the parallelization of the solver for *Jacobi* is appropriate, you should continue the parallelization process considering the dependences that appear when the *Gauss-Seidel* method is used. Recall that the dependencies arising in the Gauss-Seidel code require the application of *blocking* in order to achieve *wavefront parallelism*. To this end, the sequential code is already prepared to work by blocks. For this reason, and contrary to the course slides, the codes implementing the numerical solver provided already have 4 loops. Handling more loops has additional overhead, and is not the most efficient in terms of execution time when not needed, as is the case of both the sequential code and the *Jacobi* solver, which does not require *blocking*. However, they are provided in this way in order to ease the parallelization of the *Gauss-Seidel* solver, which requires *blocking*.

As in the previous chapter, you have to make sure that all your paralel codes work correctly by submitting the `submit-omp.sh` to the execution queue and checking if the resulting image is correct. Do not forget to specify the *Gauss-Seidel* solver when submitting the script (passing a 1 instead of a 0), as well as the number of threads to use.

For the analysis of each version implemented in this chapter you can use the tools introduced in the previous chapter, namely `strong scalability test`, `modelfactors` and `Paraver`. Thus, after validation of correctness, you should show the strong scalability achieved by using `submit-strong-omp.sh`; and explain the quality of the parallelization using the results generated by submitting `submit-strong-extrae.sh` to the `execution` queue.

In addition, we are interested in observing the usage of the *memory hierarchy*. Thus, we ask you to perform a memory analysis of the Gauss-Seidel parallel codes. We want you to observe the impact in memory hierarchy exploitation of your parallel code implementing a `Task decomposition` and at least one of your codes implementing a `Data decomposition`.

Memory Analysis: For each parallel code, you should submit the `submit-strong-memory.sh` script. This will generate several traces in a directory called `nameprogram-strong-memory` and a file: `l2_misses_...txt`. Those files contain the overall number of cache misses (the accumulation for all the threads) and the average number of misses per thread of the second level of cache. Include this information in the report. Compare your Gauss-Seidel implementations based on the number of `l2_misses` and try to reason the differences among them. In summary, use the command `sbatch submit-strong-memory.sh heat-omp 1` inspect the output file `l2_misses-heat-omp-boada-18.txt` (or boada-19 or boada-20) and reason about the locality properties of the code parallelized with implicit and the one with explicit tasks.

5.1 Task decomposition: Implementation with `explicit tasks with dependences`

In this section we ask you to parallelize the Gauss-Seidel code using **explicit** *tasks with dependencies* (`task + depend (in, out, inout)`). Validate correctness and analyze the parallel performance of your code.

For the deliverable: Include a code excerpt showing the parallelization with `task + depend (in, out, inout)`; the scalability test with a brief explanation; comment the important metrics shown by `modelfactors` which explain the parallel performance; and some captures of relavant information shown by `Paraver` which help in visualizing the reasons behing the parallel performance achieved. Include a summary of relevant information about the memory analysis.

5.2 Data decomposition: Implementation with `do-across`

In this section we ask you to parallelize using a *do-across*, i.e. by means of the `ordered + depend (sink, source)` `OpenMP` clauses. The parallelisation should follow the same *geometric block data decomposition by rows* that is shown in Figure 4.1, and again ONLY making use of the **implicit tasks** in parallel regions. However, in this case *blocking* must be implemented, resulting in the geometric block decomposition suggested in Figure 5.1.

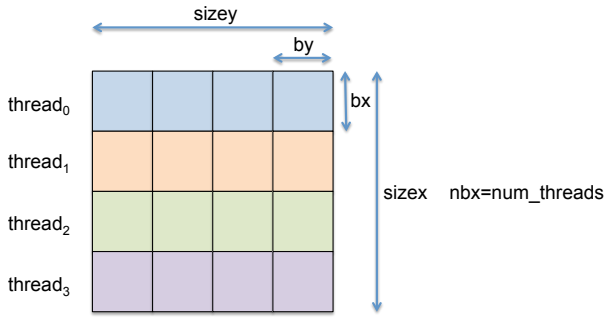


Figure 5.1: Block data decomposition by rows of the input & output matrix with blocking in the columns.

Validate correctness and analyze the parallel performance of your code.
For the deliverable: Include a code excerpt showing the parallelization with `ordered + depend (sink, source)` `OpenMP` clauses; its scalability test with a brief explanation; comment the important metrics shown by `modelfactors` which explain the parallel performance; and some captures of relavant information shown by `Paraver` which help in visualizing the reasons behing the parallel performance achieved.

5.3 Detailed Analysis and Optimization

In order to exploit more parallelism in the execution of the solver, you should change the number of blocks in the `j` dimension. Why? Changing the number of blocks in the `j` dimension changes the ratio between computation and synchronisation, and may increase the parallelism, why? In order to do this change in your code, `main` is already prepared to receive an argument in the command-line execution (`-u value`) that is stored in global variable `userparam`. Modify your code accordingly to make use of this value and change the number of blocks in the `j` dimension without recompiling the code for each new value to test. Use the value in `userparam` as a factor of the number of blocks `nblocksi` to set `nblocksj` (i.e. `nblocksj=nblocksi*userparam`). Compile and execute with some different values and check that it returns the correct results and has the expected effect.
For the deliverable: Include an excerpt of the code to show the modifications done.

5.3.1 Finding the appropriate value for the number of blocks

Use the provided `submit-userparam-omp.sh` script to explore a range of values of the number of blocks. The only argument for this script is the number of threads to be used for the exploration. Submit this script for an execution with 16 threads. In view of the results, select a suitable number of blocks. You will need it in the next subsection.

For the deliverable: Include the plot obtained with `submit-userparam-omp.sh`. Reason *why* changing the number of blocks in the `j` dimension changes the ratio between computation and synchronisation.

5.3.2 Overall Analysis

Once correctness of this version of `heat-omp` is validated and an appropriate value for the number of blocks determined, edit `submit-strong-omp.sh` and initialize variable `userparam` using the value chosen in the previous subsction. Next, run an execution with 16 threads. To this end, submit the execution of the `submit-strong-omp.sh` script to the execution queue and obtain the scalability plot; do not forget to specify with an argument the solver to be used (1). In addition, edit `submit-strong-extrae.sh` and set `userparam` with the same value above. Submit this script to the queue and analyze the tables provided by `modelfactors` as well as the trace corresponding to the execution with 16 threads using `paraver`.
For the deliverable: Include the scalability plot obtained with `submit-strong-omp.sh`. Do you observe a linear speedup? Reason your answer based on the information obtained with `modelfactors` and `paraver`.

5.4 Memory analysis

For parallel Gauss-Seidel, compare the cache misses of your parallel code implemented with `Task decomposition` and the one implementing a `Data decomposition`. Are there substantial differences in the usage of the memory hierarchy? Do they translate in differences in performance?

6

Deliverable

You have to deliver **TWO files**: 1) a `PDF` that contains your **report** explaining the parallelization of the heat equation code and speedups obtained; and 2) a compressed file `zip`) with the parallel **C source codes** and no other files. File names must clearly show which version of the parallel codes they contain. In the `PDF` file state clearly the name of all components of the group. Only one submission per group must be done through the Raco website.

7

Annex 1: Creating your own synchronisation objects

The implicit tasks used to express parallelisation strategies based on data decomposition can not synchronise themselves using task dependences (i.e. `depend` clauses can not be applied to implicit tasks). For this reason in this annex we show how to implement your own synchronisation objects to enforce some sort of task ordering constraints. These synchronisation objects will be implemented using shared variables for which one has to ensure that all accesses (reads and writes) to them always access to memory. This is what is called the *memory consistency problem*: usually the compiler tries to optimise memory accesses by placing variables in registers, and then only read/write from/to memory at certain points in the parallel program, usually at synchronisation points.

For example consider the simple producer-consumer code shown in Figure 7.1, using implicit tasks. Notice that all implicit tasks can do their *computation A* part in parallel; however, the instance of the implicit task executed by `myid` cannot execute *computation B* until the previous thread has executed it (in other words, the execution of *computation B* has to be done in an ordered way). Only the instance of the implicit task executed by thread 0 can do the execution of *computation B* initially. This is controlled by vector `next` with as many positions as threads (i.e. instances of the implicit task): if position `myid` is 0, then the implicit task will wait in the `while` loop; once it is set to 1 by the implicit task `myid-1`, the implicit task will be allowed to continue.

```
int next[P];
...
next[0] = 1;
for (int i = 1; i < P; i++) next[i] = 0;
...
#pragma omp parallel num_threads(P)
{
    int myid = omp_get_thread_num();
    // computation A

    while (next[myid] == 0); // wait to advance

    // computation B

    if (myid < P-1) next[myid+1]++;
}
```

Figure 7.1: Simple producer-consumer code.

Although the code seems to be correct, it has memory consistency problems. To ensure that each task always accesses to the last value of the shared variable `next`, the programmer has to introduce some sort of data sharing/synchronisation construct. The preferred one is `atomic`, which can have several different clauses, as for instance: `read`, `write` and `update`. Clauses `read` and `write` are used to express that memory accesses are always served by main memory. The resulting code is shown in Figure 7.2.

Inside the `do while` construct we are ensuring that the element of vector `next` is read from memory by adding the `atomic read`; in order to ensure that the element of vector `next` is updated in memory we need to add `atomic write`.

```
int next[P];
...
next[0] = 1;
for (int i = 1; i < P; i++) next[i] = 0;
...
#pragma omp parallel num_threads(P)
{
    int myid = omp_get_thread_num();
    // computation A

    do {
        #pragma omp atomic read
        tmp = next[myid];
    } while (tmp == 0); // wait to advance

    // computation B

    if (myid < P-1) {
        #pragma omp atomic write
        next[myid+1] = 1;
    }
}
```

Figure 7.2: Simple producer-consumer code making use of atomic pragmas to guarantee memory consistency.

How would you change the code above if we introduce an iterative loop that repeats the execution of *computation A* and *computation B* several times, always ensuring the same execution order constraints? Figure 7.3 shows the code to be completed in order to ensure the appropriate execution ordering.

```
int next[P];
...
next[0] = ...;
for (int i = 1; i < P; i++) next[i] = ...;
...
#pragma omp parallel num_threads(P)
{
    int myid = omp_get_thread_num();
    for (int iters = 0; iters < num_iters; iters++) {
        // computation A

        do { ... } while ( ... ); // wait to advance

        // computation B

        if (myid < P-1) next[myid+1] = ...;
    }
}
```

Figure 7.3: Second version of simple producer-consumer code.

Question: Can the access to vector `next` cause *false sharing*? If you answered yes, how would you solve the problem?